SIMATS ENGINEERING

Department of Information Technology

CSA3711 – Software Testing

CODE OF CONDUCT FOR ASSESSMENT SUBMISSION

Every student must read and follow this code of conduct.

1. The work submitted is my own individual work; this assignment is of an individual nature.
2. I have not copied from another student, from previous batches or from any other source without proper acknowledgement.
3. Any book, website, tool or software used for assistance has been acknowledged, and I understand the content I submit and can explain it in a viva.
4. I have not shared my solution with any other student and have not allowed anyone to copy it.
5. I have not fabricated or altered data, calculations or references. All calculations are shown and can be reproduced.
6. I will follow the submission format prescribed by the department: Times New Roman, 1.5 line spacing, answers written after each question.
7. I will submit the assignment on or before the due date; late submissions are subject to the rules of the department.
8. I accept that any violation of this code may lead to reduction or cancellation of marks and disciplinary action as per institutional rules.

Student Declaration

I have read the code of conduct and declare that this submission follows it.

Student Name	Rakcha A	**Register Number**	192311421
Department / Section	CSE	**Academic Year / Semester**	IV Year
Faculty Name	Kesavan R	**Nature of Assignment**	Individual
Duration / Total Marks	1 Week / 50	**Date of Submission**	

Rakcha A **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

Software Testing (CSA3711)

ASSIGNMENT 1 – CO1

CO1: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks – 50)

Duration: 1 week

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident
I1	Two patients receive the same token number for the same doctor when they book at the same second.
I2	Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.
I3	Consultation fee shown as 500 on the booking page but 550 is charged; the GST rule was misunderstood.
I4	Application crashes when a patient name has more than 40 characters.
I5	Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.
I6	Prescription PDF shows the previous patient's details when two doctors save at the same time.
I7	The user manual tells patients to cancel through "My Visits", but this menu does not exist.

ID	Incident
I8	A test case for refund was marked “Pass” because the tester compared the output with a wrong expected value.
I9	SMS reminders are not sent when the hospital’s SMS provider changes its API key.
I10	The doctor’s schedule screen takes 25 seconds to load on the hospital’s Wi-Fi network.

Defect data for the last release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix per defect (₹)	Remarks
Requirements	30	2	500	No formal review
Design	22	3	1,500	No formal review
Coding	38	15	3,000	Developer self-check only
Testing	5	20	8,000	Demo-based testing
Documentation	5	0	–	–
Production (after release)	–	60	40,000	Reported by patients

Design Challenge:

The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system. Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

Task 1 – Evaluate Testing as an Engineering Activity 5 Marks

CO1 | Bloom's Level: K5 – Evaluate

Examine whether the way MediQueue was tested qualifies as an engineering activity.

Did the MediQueue team deliberately engineer quality into the product, or did they simply hope it would be there?

Discuss:

- The features of testing as an engineering activity (planning, measurement, standards, repeatability)
- Which of these features were absent in the MediQueue project, supported by evidence from the case
- How the absence of each feature led to the incidents that were reported
- What an engineered approach to testing would have looked like for this project

You are required to assess the project against engineering practice rather than just define the term.

Evidence Expected

A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.

Was MediQueue testing an engineering activity?

My judgement is that MediQueue's testing was **not** an engineering activity. Engineering means that quality is planned, measured, governed by standards and can be repeated by someone else. The MediQueue team had none of these. Developers "tested" their own code shortly before release and the release was approved because a demo showed no errors. A demo shows only that the paths the presenter chose to show work; it says nothing about the paths nobody exercised. The team therefore hoped for quality rather than engineering it.

Table 1: MediQueue testing compared with the features of an engineering activity

Feature	What engineering practice expects	Evidence from the case	Incidents it led to
Planning	Test goals, levels, risks, schedule, resources and exit criteria written down before testing starts	No test plan. Testing was done "just before release", after eight months of construction, and no tester or time was allocated	I1, I6, I9, I10 (concurrency, interface and performance tests were never planned)
Measurement	Coverage, defects per phase and cost are measured and used for the release decision	Release criterion was "the demo showed no errors". Only 40 of 100 defects were found before release; patients found 60. No target existed to show that this was unacceptable	All ten; explains why 60% of defects escaped
Standards and defined process	Requirement and design reviews, a standard test-case format, traceability to requirements	Requirements and design had "no formal review"; coding had a "developer self-check only". The Sunday rule and GST rule were never documented	I2, I3, I7
Repeatability	Documented test cases, data and expected results that anyone can re-run after each change	Demo-based testing cannot be repeated. The one documented test mentioned (refund) was passed against a wrong expected value	I4, I5, I8
Independence and roles (supporting feature)	A person whose job is to find defects and who has testing skill	No tester existed; developers checked their own code, so the same wrong assumption was used to write and to "test" it	I1, I5, I6

How the absence of each feature led to the incidents

- **No planning:** nothing prompted the questions "what if two users book in the same second?", "what if the SMS key changes?", "how fast is the schedule screen on hospital Wi-Fi?". I1, I6, I9 and I10 are exactly the defects that appear only when such tests are planned.
- **No measurement:** without an exit criterion the team could not see the risk. Before release only 5 of the 52 requirement and design defects (2 + 3) had been found, and testing found 20 of the 75 defects then present (26.7%).

- **No standards or reviews:** a missing business rule (I2) and a misread tax rule (I3) can be found only by checking the requirement with the people who own the rule. The wrong menu name in the manual (I7) comes from the same absence of document review.
- **No repeatability:** without written cases and expected values, boundary cases such as a 41-character name (I4) or a patient aged exactly 60 (I5) are checked only by chance. When a case was written (refund), its expected value was wrong and nobody re-verified it (I8).
- **No independent tester:** the person who writes a piece of code carries the same assumptions into testing it. A developer who believes that "two doctors never save together" will never write that test (I6).

What an engineered testing approach would have looked like

- A short test policy and a test plan for every release stating goals, test levels (unit, integration, system, acceptance) and schedule.
- Release exit criteria, for example: all planned high-risk tests executed, no open Critical or High defect, at least 95% of planned tests passed.
- Reviews of requirements (with billing and OPD staff) and of the design (concurrency and data separation) before coding.
- Test cases derived by equivalence classes and boundary values, each with an expected result taken from the requirement and traced back to it.
- System tests for concurrency, performance on the hospital Wi-Fi and the SMS interface, run in a proper test environment.
- A defect repository with simple measures (defects per phase, cost, escapes) reviewed at each release.

Conclusion: testing in MediQueue was an event, not an engineering process. Every missing feature maps to at least one incident, so the incidents were not bad luck; they were the predictable result of how the work was organised.

Task 2 – Evaluate the Role of Process in Software Quality 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Hospital management is convinced that employing "better developers" will fix the quality problem.

Does quality come mainly from good people or from a good process?

Discuss:

- How a defined process contributes to achieving software quality
- Whether better developers on their own would have prevented the incidents
- Where a process maturity view (for example, TMM levels) would position the current team

You must defend your position using evidence from the case.

Evidence Expected

A well-justified position on how process affects quality, backed by data from the case.

Good people or good process?

My position is that MediQueue's problem is **mainly a process problem**. Good developers help, but most of the incidents arose before coding or outside the code, where developer skill cannot reach. A defined process acts on every phase (reviews, planned tests, an independent tester, a repository) and also catches the mistakes that good developers still make.

Table 2: Could a better developer alone have prevented the incident?

Incident	Prevented by a better developer alone?	Reason
I1, I6	Possibly	An experienced developer may think of concurrency, but a design review and a concurrency test are still needed to be sure
I4, I5	Probably	Input length and age boundaries are coding skills, yet a boundary test catches them even with an average developer
I10	Partly	Also needs a performance target (requirement) and a load test
I2, I3	No	The Sunday rule was never documented and the GST rule was misread; coding skill cannot supply a rule that was never given
I7	No	A documentation defect; needs a document review
I8	No	A defect in testing; needs test-case review and an independent tester

Incident	Prevented by a better developer alone?	Reason
I9	No	The SMS provider changed its key; needs interface monitoring and configuration control

Skill alone could have prevented at most 4 or 5 of the 10 incidents. The defect data agree: only 38 of the 100 defects were introduced in coding, while 52 came from requirements and design. Even if better developers halved coding defects, 19 defects would disappear and 81 would remain, so hiring better developers addresses less than a fifth of the problem.

Position of the team on a process maturity scale

On the Testing Maturity Model (TMM) I place the team at **Level 1 (Initial)**: testing is not separated from debugging, its only aim is to show that the software works (the demo), and there is no test plan or trained tester. The next target is **Level 2 (Phase Definition)**, whose goals fit exactly what is missing: defined testing and debugging goals, test planning and basic testing techniques. Level 3 (Integration) would follow, with a test group, a test life cycle integrated with development, and training.

Task 3 – Evaluate Testing as a Process 6 Marks

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was carried out as one activity immediately before release.

How would the defect data have differed if testing had been treated as a process spanning the entire life cycle?

Discuss:

- Testing as a process made up of planning, design, execution, evaluation and reporting activities
- How verification and validation activities relate to each development phase
- The phase in which each kind of MediQueue defect ought to have been detected
- Using the defect data table, the cost the hospital incurred because defects were discovered late
- The probable saving had requirements and design reviews been carried out
- The limitations of your estimate and the assumptions on which it rests

Show your calculations and explain what they mean. Merely stating that "early testing is cheaper" will not earn full marks.

Evidence Expected

A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.

1. Testing viewed as a process

Table 3: Test process activities applied to MediQueue

Activity	What it would involve in MediQueue	Output
Planning	Define test goals, levels, risks (concurrency, money, privacy), schedule and exit criteria	Test plan
Design	Derive cases from requirements: age 59/60/61, name length 40/41/42, Sunday booking, fee displayed = fee charged, two bookings in the same second	Test cases with expected results
Set-up	Test bed with test doctors and patients, an SMS sandbox and a representative Wi-Fi network	Test environment
Execution	Run planned tests at unit, integration and system level; re-run after every fix	Test log

Activity	What it would involve in MediQueue	Output
Evaluation	Compare actual results with expected ones (the oracle), decide pass/fail, check the exit criteria	Verdicts and defect reports
Reporting and closure	Defect report, test summary and lessons learnt stored in the repository	Summary report, repository entries

2. Verification and validation in each phase

Table 4: Verification and validation activities for each development phase

Phase	Verification (building it right?)	Validation (building the right thing?)	Defects targeted
Requirements	Checklist review and testability check	Acceptance test planned with hospital staff	I2, I3, I10
Design	Design inspection (concurrency, data handling, interfaces)	System test cases designed	I1, I6, I9
Coding	Code review, coding standard, boundary-value unit tests	–	I4, I5
Integration and system test	Interface and regression tests, test-case review	System tests (performance, concurrency) and user acceptance test	I1, I3, I6, I9, I10
Documentation	Manual checked against the working system	Manual tried by a real user	I7
Operation	Interface monitoring, regression after change	Patient feedback	I9

3. Phase in which each incident should have been detected

Table 5: Intended detection phase for each incident

ID	Defect	Phase to detect	Activity that would reveal it
I1	Same token to two patients	Design	Design inspection of token allocation; concurrent-booking system test
I2	Sunday rule missing	Requirements	Requirement review with OPD staff
I3	₹500 shown, ₹550 charged	Requirements	Review with billing; system test comparing displayed and charged fee
I4	Crash for names over 40 characters	Coding	Boundary-value unit test (40, 41)

ID	Defect	Phase to detect	Activity that would reveal it
I5	No senior priority at age 60	Coding	Boundary test (59, 60, 61); code review
I6	Wrong patient on prescription PDF	Design	Design inspection of shared data; concurrent-save test
I7	Manual mentions a missing menu	Documentation	Review of manual against the application
I8	Wrong expected value in refund test	Testing	Second-person review of the test case
I9	SMS stops after key change	Design / integration	Interface test, configuration control, production monitoring
I10	Schedule screen takes 25 s	Requirements / system test	Performance requirement; load test on hospital Wi-Fi

Under such a process five incidents (I1, I2, I3, I6, I10) would be caught in requirements or design, two (I4, I5) in coding, and the remaining three in testing, documentation or operation. In the real project patients found all ten.

4. Cost of late detection

Table 6: Cost of the last release's 100 defects, by phase of detection

Phase of detection	Defects detected	Cost each (₹)	Total cost (₹)	Share of total
Requirements	2	500	1,000	0.04%
Design	3	1,500	4,500	0.17%
Coding	15	3,000	45,000	1.72%
Testing	20	8,000	1,60,000	6.13%
Production (after release)	60	40,000	24,00,000	91.93%
Total	**100**	average 26,105	**26,10,500**	**100%**

Calculation: 2 × 500 = 1,000; 3 × 1,500 = 4,500; 15 × 3,000 = 45,000; 20 × 8,000 = 1,60,000; 60 × 40,000 = 24,00,000. Sum = ₹26,10,500, an average of ₹26,105 per defect.

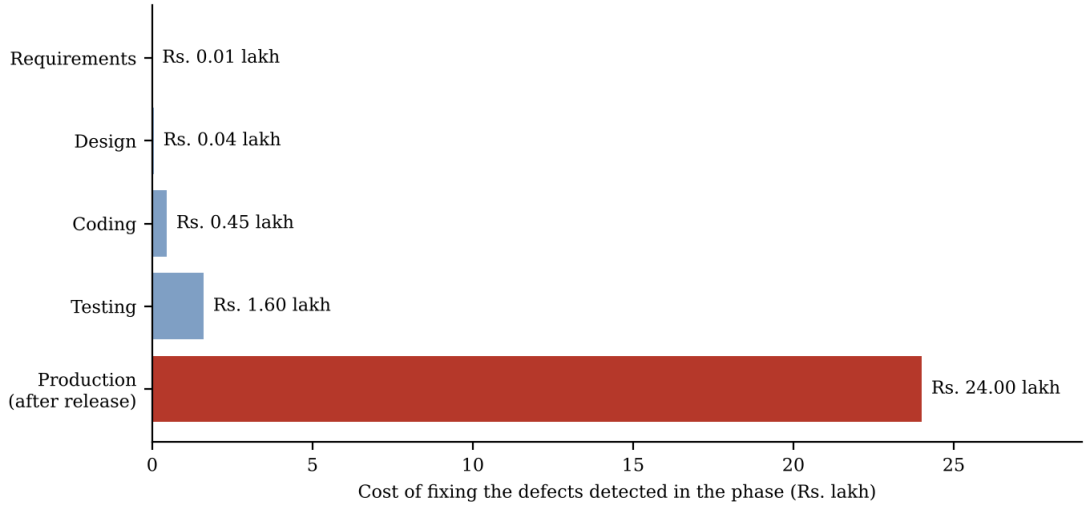


Figure 1: Cost of defects by the phase in which they were detected

Interpretation. Patients found 60% of the defects and these account for 91.9% of the cost. One production defect costs 80 times a requirements defect (40,000 ÷ 500) and 5 times a testing defect (40,000 ÷ 8,000). If the 60 production defects had been caught in testing at ₹8,000 each they would have cost ₹4,80,000 instead of ₹24,00,000, a difference of ₹19,20,000 (an upper limit, since testing could not catch all of them). If each of the 95 defects with a known unit cost had been fixed in the phase that introduced it, the cost would have been 30 × 500 + 22 × 1,500 + 38 × 3,000 + 5 × 8,000 = ₹2,02,000. Late detection therefore cost about ₹24 lakh (26,10,500 − 2,02,000), roughly twelve times the ideal cost.

Table 7: Defect flow: defects caught by each phase out of those present

Phase	Introduced	Present in phase	Detected	Detection efficiency	Passed on
Requirements	30	30	2	6.7%	28
Design	22	50	3	6.0%	47
Coding	38	85	15	17.6%	70
Testing	5	75	20	26.7%	55
Documentation	5	60	0	0.0%	60
Production	–	60	60	100%	0

Present = carried over from the previous phase + newly introduced. The 60 defects left after documentation equal the 60 found in production. Requirements and design introduced 52 defects but detected only 5 (9.6%). Only 40 of 100 defects were found before release, so “no errors in the demo” was concluded while 60 defects were still hidden.

5. Probable saving from requirements and design reviews

Using the flow in Table 7, I assume that a requirements review and a design review each remove a fraction r of the defects present in their phase (instead of 6.7% and 6.0%), while coding and testing keep their present detection rates (17.6% and 26.7%). Defects removed early are charged at that phase's cost; the rest move on. I tested r = 40%, 60% and 80%, and assumed the two review programmes cost ₹1,00,000 (about 125 person-hours at ₹800 per hour).

Table 8: Estimated savings from requirements and design reviews

Review efficiency	Found by reviews	Reaching patients	Total cost after reviews (₹)	Gross saving (₹)	Review cost (₹)	Net saving (₹, % of 26,10,500)
40%	28.0	46.1	20,26,808	5,83,692	1,00,000	4,83,692 (18.5%)
60%	38.4	39.8	17,61,399	8,49,101	1,00,000	7,49,101 (28.7%)
80%	46.4	35.0	15,55,854	10,54,646	1,00,000	9,54,646 (36.6%)

At 60% review efficiency the defects reaching patients fall from 60 to about 40, cost falls to about ₹17.6 lakh and the net saving is about ₹7.5 lakh (28.7%) after paying for the reviews. Even at 40% the reviews pay off (net ₹4.8 lakh), because ₹1 lakh of review effort avoids many ₹40,000 production fixes. In the same model, a test process catching 60% of the defects present in the testing phase (instead of 26.7%) would cut production defects from 60 to about 35, so reviews and better testing complement each other.

6. Limitations of the estimate and its assumptions

- **Defect flow:** the case does not say in which phase each detected defect was introduced; I assumed that missed defects move to the next phase.
- **Review efficiency:** the 40–80% range is assumed because the team has no review data; poorly run reviews would be at the low end.
- **Unit costs:** the case gives rework cost only. Patient harm, lost trust, legal exposure and hospital staff time are excluded, so the true cost of production defects is higher.
- **Documentation defects:** no cost is given; I treated the five left at the end as production defects.
- **One release, one team:** 100 defects from one project; defects never discovered are not included.
- **Review cost:** ₹1,00,000 is my own estimate; a higher cost would reduce, but not remove, the benefit.

Task 4 – Apply and Critique Basic Definitions 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

In its reports, the team uses the terms “bug”, “error” and “failure” as if they mean the same thing.

Does loose terminology genuinely damage quality, or is it merely a question of words?

Discuss:

- What error, fault (defect), failure, test case, test oracle and test bed mean in the MediQueue context
- The error, the fault and the failure for incidents I3, I5 and I6
- How the confusion over terminology allowed incident I8 to happen
- Whether the team requires a shared glossary, and what it should include

Definitions reproduced from a textbook will not earn full marks. Each one must be applied to MediQueue.

Evidence Expected

Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.

Terminology is more than words

If “bug”, “error” and “failure” all mean the same thing, nobody can tell whether the problem lies with a person, in the product or in what the user saw, and so the cause cannot be traced or prevented. I first apply the terms to MediQueue and then to the incidents.

Table 9: Basic definitions in the MediQueue context

Term	Meaning	MediQueue example
Error	A mistake made by a person	A developer assumes that two doctors never save a prescription at the same time
Fault (defect, bug)	A flaw in the software or a document, caused by an error	The PDF generator keeps patient data in a variable shared by all doctors
Failure	Wrong behaviour seen by the user when the faulty code runs	The prescription PDF shows the previous patient’s details
Test case	Inputs, conditions and expected result for one check	Book a Sunday appointment → system must reject it with “OPD closed”

Term	Meaning	MediQueue example
Test oracle	The source that tells the expected result (requirement, rule, calculation, expert)	The approved fee table says fee = ₹500 + tax; it decides whether ₹550 is right
Test bed	The environment where tests run: hardware, network, data, tools, stubs	A copy of the application with test doctors and patients, an SMS sandbox and hospital Wi-Fi conditions

Table 10: Error, fault and failure for incidents I3, I5 and I6

ID	Error (human mistake)	Fault (defect in the product)	Failure (what users saw)
I3	The analyst or developer misread the GST rule and implemented it differently in two places	The booking page shows the base fee while the payment module adds tax: two separate fee calculations	Page shows ₹500 but the patient pays ₹550
I5	The developer read "senior citizen" as "over 60", or compared only the year rather than the full date	A condition such as age > 60 (or age computed on the booking date, not the appointment date)	A patient who turns 60 on the appointment day gets no priority
I6	The developer assumed that two doctors never save at the same moment	Patient details are held in shared data that the second doctor overwrites before the PDF is built	The PDF shows the previous patient's details

How terminology confusion allowed I8

In I8 a refund test was marked "Pass" because the output was compared with a wrong expected value. The tester's error created a fault in the **test case** (a wrong oracle), which produced a false verdict. A team that calls everything a "bug" does not separate a fault in the product from a fault in the test, and never asks "where did this expected value come from?". No test oracle had been defined, so the expected value was written from memory instead of from the requirement and the "Pass" was trusted. The refund defect stayed hidden: the wrong word concealed a wrong test.

Does loose terminology damage quality?

Yes, in four ways. (1) Root-cause analysis fails: a failure is caused by a fault and a fault by an error, and each needs a different remedy (a fix, a review checklist, training). (2) Measurement is distorted: the 100-defect count mixes failures reported by patients with faults in code, so one fault may be counted several times. (3) Communication with management suffers, because

“bug” says nothing about severity. (4) Fix verification is weak, because nobody records which test case and oracle prove that the fault is gone.

A common glossary

The team needs a one-page glossary agreed by developers, testers and hospital staff and kept with the defect repository.

Table 11: Minimum contents of the MediQueue testing glossary

Group	Terms to define	Why it is needed
Defect terms	Error, fault/defect, failure, incident	Separates cause, flaw and symptom in each report
Test terms	Test case, expected result, oracle, test bed, test log, regression test	Makes tests repeatable and prevents wrong expected values (I8)
Rating terms	Severity and priority levels with MediQueue examples	Ends arguments about what to fix first
Defect states	New, assigned, fixed, verified, closed, reopened	Tracks every defect to closure
Product terms	Build, release, hotfix, appointment day, senior citizen, fee, token	Removes ambiguities such as “senior at 60” (I5)

Task 5 – Evaluate the Application of Testing Principles 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The developers assert that "we tested everything and found no errors, so the software was correct".

Which testing principles show that this assertion is false?

Discuss:

- A minimum of four software testing principles relevant to the MediQueue situation
- For every principle, the incident or decision in the case that breaks or supports it
- Which principle, had it been followed, would have had the greatest impact on quality, and why

Do not just list the principles. You must link each principle to evidence.

Evidence Expected

A justified assessment of the developers' assertion using testing principles tied to evidence from the case.

Why the developers' claim is wrong

The claim makes two mistakes. It treats the absence of failures in the few tests actually run as proof of correctness, and it assumes that "everything" could be tested. Six principles, each linked to the case, show why it fails.

Table 12: Testing principles applied to MediQueue

Principle	Violated / confirmed	Evidence from the case
Testing shows the presence of defects, never their absence	Violated	The demo showed no errors, yet patients later found 60 defects; the lack of failures in a few demo cases was taken as proof of correctness
Exhaustive testing is impossible, so tests must be chosen by risk	Violated	No risk-based selection was made; concurrency, money, privacy and performance (I1, I3, I6, I10) were never targeted
Testing must be planned and start early, throughout the life cycle	Violated	No plan; requirements and design were not reviewed. 52 of 100 defects were introduced there, only 5 were caught early, and late detection cost ₹26.1 lakh, 91.9% of it in production
Test cases must cover invalid and boundary inputs	Violated	A 41-character name (I4), age exactly 60 (I5) and a Sunday booking (I2) were never tried

Principle	Violated / confirmed	Evidence from the case
Every test case must define an expected result that is checked carefully	Violated	The refund test was passed against a wrong expected value (I8)
Testing should be done by a qualified, independent person; a good test is one likely to find a new defect	Violated	Developers tested their own code, and the "test" was a management demo meant to show success, so it found nothing (I5, I6)
Defects cluster	Confirmed	I1 and I6 share one root cause (shared data under simultaneous use); after the first was found, the second should have been searched for

The principle with the largest effect

In my judgement, **"testing should start early and run through the whole life cycle"** would have had the greatest impact. Half the incidents (I1, I2, I3, I6, I10) come from requirements and design, where only 9.6% of defects were caught, and the cost per defect rises from ₹500 in requirements to ₹40,000 in production. Planning and independent testing help, but both work best when they begin in the requirements phase. Writing test cases from the requirements would have exposed the missing Sunday rule and the fee mismatch before any code existed.

Task 6 – Evaluate the Tester's Role 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

MediQueue did not have an independent tester.

Would an independent tester have altered the outcome, or would the same defects still have reached patients?

Discuss:

- The role and duties of a tester throughout the life cycle
- The benefits and drawbacks of developers testing their own code
- Which incidents a tester would most probably have detected, and at what stage
- The qualities required of a tester working on a healthcare application

Your answer must adopt a position and defend it.

Evidence Expected

A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.

Would an independent tester have made a difference?

My position: an independent tester would have **changed the outcome considerably, but not eliminated every defect**. Working across the life cycle, a tester would very probably have caught seven of the ten incidents. The other three (I1, I6, I9) also need design inspection, specialised concurrency tests and production monitoring.

The tester's role through the life cycle

- **Requirements:** review for statements that are missing, unclear or untestable; ask "what happens on Sunday? is the fee inclusive of tax? how fast must the schedule load?".
- **Design and planning:** prepare the test plan, risks and exit criteria; design test cases and the test bed.
- **Coding and integration:** support unit testing, run integration tests and maintain the regression suite.
- **System and acceptance:** run functional, load, concurrency and security tests and support user acceptance.
- **Throughout:** report defects with reproduction steps, verify fixes, maintain the repository and give management evidence for the release decision.

Developers testing their own code

Benefits: the developer knows the code, finds simple faults quickly and fixes them cheaply; developer unit testing is valuable and should continue. **Drawbacks:** the developer tests what he or she believes the code does, so a wrong assumption is carried into the test; there is a natural wish to show the work is good; deadlines push testing to the end; and nobody checks the developer's reading of the requirement. MediQueue shows this: I5 (age-60 boundary) and I6 (single-user assumption) both come from the developer's own assumptions.

Table 13: Incidents a tester would most probably have caught

ID	Detected by tester?	Stage and how
I2	Yes	Requirement review / test design: ask for the OPD working days
I3	Yes	Requirement review; system test comparing the displayed fee with the amount charged
I4	Yes	Unit or system test with boundary-length names (40 and 41)
I5	Yes	Test design with boundary values 59, 60, 61 on the appointment day
I7	Yes	Documentation review: follow the manual step by step on the real application
I8	Yes	Second-person review of the test case; trace the oracle to the requirement
I10	Yes	System load test on hospital Wi-Fi once a performance target exists
I1	Maybe	Only if a concurrency test is designed (needs tools and skill); design inspection is the stronger safeguard
I6	Maybe	As I1: needs a concurrent-save test and a design review of shared data
I9	Partly	Interface test with an SMS sandbox; a post-release key change needs monitoring and configuration control

The result is 7 "yes", 2 "maybe" and 1 "partly". In the Task 3 model, raising testing-phase detection from 26.7% to 60% would cut production defects from 60 to about 35. That is a large gain, but 35 defects would still reach patients, so a tester alone is not enough and my plan also includes reviews.

Qualities of a healthcare tester

- **Domain knowledge:** outpatient workflow, fees and tax, senior-citizen rules, prescriptions.
- **Eye for detail:** notices a ₹50 difference, a wrong menu name or a wrong expected value.

- **Sensitivity to patient safety and privacy:** understands that showing one patient's data to another (I6) is a serious breach.
- **Technical skill:** test design techniques, concurrency and load testing, defect tracking, basic automation.
- **Communication and ethics:** reports defects clearly and without blame, keeps health data confidential and can say "not ready for release".

Task 7 – Trace the Origins of Defects 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents logged after release, determine where each defect originated.

At which points in the life cycle are MediQueue's defects actually originating?

Discuss:

- The origin of every incident: requirements, design, code, testing, documentation or environment
- The evidence from the case that supports each origin you assign
- Incidents that could have more than one origin, and how you reached your decision
- What the overall pattern of origins indicates to the hospital about where effort should be invested

Where an incident could fit more than one origin, argue in favour of your choice.

Evidence Expected

A justified origin for every incident and an interpretation of the overall pattern of defect origins.

Where do the defects come from?

For each incident I assign one main origin, give the evidence, and name a possible second origin with my reason for preferring the main one.

Table 14: Origin of every incident

ID	Main origin	Evidence from the case	Second origin and why I chose the main one
I1	Design	Duplicate token occurs only when bookings fall in the same second; token allocation was not designed as an atomic operation	Coding. Design chosen because the flaw is in how allocation is structured, not a typing or logic slip
I2	Requirements	"This rule was never written in the requirements"	None
I3	Requirements	"The GST rule was misunderstood"; the rule was wrong or unclear at requirement stage	Coding. Requirements chosen because the misunderstanding concerns a business rule
I4	Coding	Crash on names over 40 characters; length handling missing	Requirements (no maximum length specified). Coding chosen because a program must not crash on any input

ID	Main origin	Evidence from the case	Second origin and why I chose the main one
I5	Coding	The age-60 rule is simple; only the boundary was coded wrongly	Requirements (date of "turning 60" not stated). Coding chosen because the rule is common knowledge
I6	Design	Two doctors overwrite shared data; a decision about shared state	Coding. Design chosen because data sharing is an architectural decision
I7	Documentation	Manual refers to a "My Visits" menu that does not exist	None
I8	Testing	Tester compared the output with a wrong expected value	None
I9	Environment / design	The SMS provider changed its API key; the application neither handled nor monitored an external change	Design (no error handling or alerting). Environment is the trigger, but the design weakness let it cause harm
I10	Requirements	No performance target appears anywhere in the case	Design or coding (slow loading). Without a target no one could call 25 s a defect

Table 15: Overall pattern of origins

Origin	Incidents	Count	Share of the 10 incidents
Requirements	I2, I3, I10	3	30%
Design	I1, I6	2	20%
Coding	I4, I5	2	20%
Testing	I8	1	10%
Documentation	I7	1	10%
Environment	I9	1	10%

Interpretation. Five of ten incidents (50%) start in requirements and design, matching the defect data, where 52 of 100 defects were introduced in these two phases. Only two incidents (20%) start in coding, yet the team's only effort (self-check) went into code. Three incidents (testing, documentation, environment) are not code defects at all. I1 and I6 share one cause, shared data under simultaneous use, which suggests the same design weakness may exist elsewhere.

For the hospital: invest first in the early phases (requirement reviews, design inspections) and in independent testing rather than in more coding skill. These are also the cheapest points for removal (₹500 and ₹1,500 per defect against ₹40,000 in production).

Task 8 – Evaluate Defect Classes and Severity 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

A scheme for classifying defects helps the team understand what kind of defects it has.

Which defect classes cause the most harm to patients, and do the team's priorities match this?

Discuss:

- An appropriate defect classification (requirements/specification, design, coding and testing defects together with their sub-classes)
- The class, severity and priority of every incident, with reasons
- Why the same defect may have different severity and priority (for example, I7 and I1)
- Which defect class the team ought to address first, and why

Just placing defects in a table is not enough. You must justify every classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.

Classification scheme

I use the four standard classes with their sub-classes. Two incidents fall outside them (the user manual and the external SMS provider), so I add a Documentation class and treat the external provider as an external-interface sub-class of Design.

Table 16: Defect classification scheme applied to MediQueue

Class	Sub-classes	MediQueue examples
Requirements / specification	Functional description, feature, business rule, interface description, non-functional (performance)	I2, I3, I10
Design	Algorithmic/processing, control logic and sequence, data and shared state, module interface, external interface	I1, I6, I9
Coding	Algorithmic/processing, control logic (boundary), data/input handling, initialisation, typographical	I4, I5
Testing	Test-case design, test oracle (expected result), test procedure, test harness	I8
Documentation (added)	User manual, help text, release notes	I7

Scales used. Severity = impact on patients and the hospital: Critical (risk to patient safety or privacy, or money lost by many patients), High (key function wrong or unusable), Medium (function wrong but a workaround exists), Low (cosmetic or informational). Priority = the order of fixing, based on severity together with frequency, workaround, cost of fix and risk of delay.

Table 17: Class, severity and priority of every incident

ID	Class / sub-class	Severity	Priority	Reason
I1	Design / control logic (concurrency)	High	Medium	Two patients sharing a token disrupts the queue, but it needs the same second and same doctor and reception can issue a new token
I2	Requirements / business rule (missing)	Medium	High	Patients travel to a closed OPD but safety is not affected; the fix is an inexpensive rule and many people are affected
I3	Requirements / business rule (wrong)	High	Critical (first)	Patients are overcharged on every booking: lost money and trust, possible legal and tax issues
I4	Coding / input handling	High	Medium	The app crashes, but only for names over 40 characters; a workaround (shorter name) exists and the fix is easy
I5	Coding / control logic (boundary)	Medium	Medium	A small group loses priority on its birthday; a fairness issue, not a safety one
I6	Design / data and shared state	Critical	Critical (first)	One patient's prescription shows another patient's details: a breach of patient safety and privacy
I7	Documentation / user manual	Low	High	No data affected, but every patient following the manual is misled and the fix takes minutes
I8	Testing / test oracle	High	High	The false "Pass" hides a possible refund defect and casts doubt on all test results; money is at stake
I9	Design / external interface	Medium	Medium	Missed reminders cause missed appointments but no harm; needs alerting and key handling
I10	Requirements / non-functional (performance)	Medium	Medium	Doctors' screen is slow but accurate; needs a target and profiling, so it is planned for the next cycle

Why severity and priority can differ

Severity is the damage if the defect occurs; priority is how soon it should be fixed, which also depends on frequency, workaround, cost of fix and what else is waiting. I7 is Low in severity

but High in priority: it damages no data, yet every patient reading the manual is affected and the fix takes minutes. I1 is High in severity but only Medium in priority: the impact is serious, but it needs two bookings in the same second and reception can issue a new token, so with I3 and I6 waiting it comes after them. A priority list based on severity alone would therefore be wrong.

Most harmful class and where to begin

The most harmful defects are the design defect on shared data (I6, Critical) and the requirement defect involving money (I3, Critical). The team's own priorities did not reflect this: it saw only visible errors in a demo and had no severity scale, so a crash and a privacy leak would have been treated alike.

I recommend addressing the **requirements/specification class first**, for three reasons: (1) it accounts for 30 of the 100 defects of the last release and 3 of the 10 incidents; (2) only 2 of those 30 were caught before release (6.7%); (3) it is the cheapest class to fix early (₹500 per defect against ₹40,000 in production). Design inspection of shared data and concurrency should follow immediately, because that class contains the Critical defect I6.

Task 9 – Design and Appraise a Defect Repository 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

At present the hospital logs complaints in e-mails and a spreadsheet.

What information should a defect repository store so that the team can learn from its defects and avoid repeating them?

Discuss:

- The fields a MediQueue defect repository should include, and why each one is required
- How the repository supports defect analysis, process improvement and prevention
- The shortcomings of the present e-mail and spreadsheet method

You must justify your repository design rather than merely listing fields.

Evidence Expected

A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.

A defect repository for MediQueue

A repository is the project's memory. For each defect it must let the team answer: what went wrong, where it began, why it was not caught earlier, how serious it was, how it was fixed and how to stop it recurring.

Table 18: MediQueue defect repository fields and the reason for each

Field group	Fields	Reason
Identification	Defect ID, title, date reported, reporter, source (patient complaint, tester, review, monitoring)	Unique reference; shows who finds defects and how, so weak detection becomes visible
Description	Description, reproduction steps, expected result, actual result, attachments	Lets a developer reproduce and fix without questions; separates the failure from expected behaviour
Location	Module/feature, release and build, environment, browser/device	Shows where defects cluster and which release holds them
Classification	Class and sub-class (requirements, design, coding, testing, documentation, environment)	Basis for analysing defect types and building checklists

Field group	Fields	Reason
Origin and detection	Phase introduced, phase detected, detection method	Gives the phase-containment figures used in Task 3 and the escape rate
Rating	Severity, priority, safety/privacy flag, number of patients affected	Sets the order of work and highlights serious risks
Handling	Status (new, assigned, fixed, verified, closed, reopened), owner, dates	Tracks each defect to closure and supports ageing reports
Cause and fix	Root cause (error), description of fix, effort (hours), cost	Allows cost-per-phase figures and learning from causes
Traceability	Related requirement ID, test case ID, related/duplicate defects	Shows which requirement or test was missing and links recurring defects
Prevention	Preventive action, owner, status	Turns each defect into a process improvement

How the repository supports analysis, improvement and prevention

- **Analysis:** Pareto charts by class, module and phase show where to focus; origin and detection counts give the Task 3 figures for every release.
- **Process improvement:** a growing share of requirement defects found late shows that reviews are ineffective; many reopened defects point to weak fixing; the data are used to set targets (for example production escapes below 20%).
- **Prevention:** each cause becomes an action (a checklist item, a coding rule, a regression test). I1 and I6 would add “shared data under simultaneous use” to the design checklist and a concurrency test to every release.
- **Test design and management:** past defects supply regression cases and new test ideas, and trend data replace “the demo went well” in release decisions.

Shortcomings of the e-mail and spreadsheet method

Table 19: E-mail and spreadsheet compared with a repository

Aspect	E-mails and spreadsheet	Defect repository
Structure	Free text; people use different words and columns	Fixed fields and lists; common terms
Duplicates	Same problem reported again and again without being noticed	Duplicates linked to one defect
Workflow	No status or owner; defects are lost in mailboxes	Status, owner, due dates, reminders

Aspect	E-mails and spreadsheet	Defect repository
History and audit	No change history; sheet versions conflict	Every change logged with date and user
Analysis	Counting by class, phase or cost needs manual work	Built-in reports and filters
Traceability	No link to requirement, test or build	Linked to all three
Privacy	Patient names and health details circulate in e-mail and shared sheets	Access control and masking of patient data

In my judgement the repository pays for itself. It can be a free or low-cost tool, the fields add only a few minutes per defect, and in return the hospital gets the data it needs to decide where to invest, which it could not get before.

Task 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Choose three incidents that belong to different defect classes.

For each chosen defect, which single process change would have prevented it?

Discuss:

- The root cause of each chosen defect
- A prevention strategy for each one (for example, requirement reviews, design inspection, coding standards, test oracles)
- The cost and effort of each strategy weighed against its benefit
- Which strategy offers the hospital the best value

Your recommendation must be backed by evidence from the case and the defect data.

Evidence Expected

A root-cause analysis of three defects with justified prevention strategies that take cost into account.

Root causes and prevention strategies

I chose one incident from each of three classes: **I3** (requirements), **I6** (design) and **I5** (coding).

Table 20: Root cause and prevention strategy for the chosen defects

Incident (class)	Root cause	Prevention (one process change)
I3 – wrong fee charged (requirements)	The fee and tax rule was never documented or agreed with billing; it was implemented twice (display and payment) from different readings	Requirement review with billing staff and a written fee-rule table; calculate the fee in one place only; use the table as the test oracle
I6 – wrong patient on prescription (design)	Design shares data across users and nobody checked it for simultaneous use; no design review took place	Design inspection with the checklist item "shared data and simultaneous users", plus a concurrency test for saving prescriptions
I5 – no senior priority on 60th birthday (coding)	Age rule coded without a boundary check; no one read the code or tested 59/60/61	Coding standard for date and age calculation, code review and boundary-value unit tests (59, 60, 61)

Cost assumptions. One person-hour = ₹800. A defect that escapes its own phase costs on average ₹32,000 to fix later, which is the mean of the testing and production figures in the case: (1,60,000 + 24,00,000) ÷ 80 defects = ₹32,000. Extra defects caught early come from the Task

3 model (reviews at 60% efficiency) and, for coding, from raising coding-phase detection from 17.6% to 35% of the 85 defects present.

Table 21: Cost and benefit of each strategy for one release

Strategy	Effort (hours)	Cost (₹)	Extra defects caught early	Saving per defect (₹)	Total saving (₹)	Benefit ÷ cost
Requirement review	50	40,000	16 (18 instead of 2)	31,500 (32,000 − 500)	5,04,000	12.6
Design inspection	75	60,000	17.4 (20.4 instead of 3)	30,500 (32,000 − 1,500)	5,30,700	8.8
Coding standard + code review + unit tests	80	64,000	14.8 (29.8 instead of 15)	29,000 (32,000 − 3,000)	4,27,750	6.7

Each strategy returns more than six times its cost. The savings overlap (a defect can be caught only once), so they cannot simply be added, but the ranking is clear.

Best value: the requirement review. It has the highest benefit-to-cost ratio (12.6), needs only discussion with billing and OPD staff, needs no tools, and attacks the largest group of defects (30 of 100). It would have prevented I2, I3 and I10 and supports the other strategies, because test oracles are drawn from the reviewed requirements. The design inspection has a lower ratio (8.8) but I would keep it, because it addresses the Critical defect I6, where one occurrence (a privacy breach) costs far more than the ₹40,000 assumed in the model.

Task 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Drawing on your analysis in Tasks 1–10, propose a quality improvement plan for MediQueue.

Discuss:

- The process changes you propose, in order of priority
- How testers, reviews and the defect repository fit into your plan
- The expected impact on the number and cost of defects
- The trade-offs, risks and limitations of your plan
- Whether the plan is still workable for a small team of five developers

Your recommendation must arise from your own analysis. A generic list of best practices will not be enough.

Evidence Expected

A technically justified quality improvement plan backed by the evidence built up in earlier tasks.

Quality improvement plan for MediQueue

The plan follows the evidence of Tasks 1–10: half the incidents (and 52% of defects) begin in requirements and design; only 9.6% of these are caught early; production defects carry 91.9% of the cost; and there is no independent tester, no exit criterion and no repository. Changes are ranked by value for money and by what five developers can realistically absorb.

Table 22: Quality improvement plan in priority order

No.	Change	Role of tester / reviews / repository	Addresses	Effort
1	Release gate: documented exit criteria (no open Critical/High defect, planned high-risk tests done, at least 95% passed) signed off by the hospital	Tester assembles the evidence; management decides	Demo-based approval; all incidents	A few hours per release
2	Review of every rule (fees, working days, age rules, performance target) with billing and OPD staff; keep a business-rule table	Tester and developers review; the table is the test oracle	I2, I3, I5, I10	About 50 person-hours

No.	Change	Role of tester / reviews / repository	Addresses	Effort
3	Appoint an independent tester (full-time, or a trained hospital IT staff member)	Writes the test plan, designs tests by equivalence classes and boundaries, runs system tests, reviews test cases	I4, I5, I7, I8, I10	About 8 months of one person
4	Design inspection of high-risk areas: shared data, concurrency, external interfaces; add concurrency and load tests	Tester and two developers; checklist kept in the repository	I1, I6, I9	About 75 person-hours
5	Defect repository with the Task 9 fields; monthly review by class, phase and cost	Whole team; tester owns the reports	Learning from every incident	1 day set-up, 5 minutes per defect
6	Coding standard, peer review and boundary-value unit tests	Developers	I4, I5	About 80 person-hours
7	Manual walk-through on the working application; monitoring and alerting for the SMS provider	Tester, operations	I7, I9	A few hours

Expected impact on the number and cost of defects

I used the Task 3 flow model with these assumptions: requirement and design reviews remove 60% of defects present; coding-phase detection rises from 17.6% to 35%; testing-phase detection rises from 26.7% to 60%. Plan costs: tester ₹50,000 per month × 8 = ₹4,00,000; reviews ₹1,00,000; code review and unit tests ₹64,000; training and tools ₹50,000.

Table 23: Expected outcome for one release of the same size as the last one

Measure	Last release	With the plan
Defects detected before release	40	79.6
Defects reaching patients	60	20.4
Cost of fixing the defects (₹)	26,10,500	10,95,412
Cost of the plan (₹)	–	6,14,000
Total cost (₹)	26,10,500	17,09,412
Net saving (₹)	–	**9,01,088 (34.5%)**

The plan cuts defects reaching patients by about two thirds (60 to about 20) and saves about ₹9 lakh per release after paying for itself, without counting unquantified benefits such as fewer patient-safety and privacy incidents.

Trade-offs, risks and limitations

- **Slower first release:** reviews and testing use calendar time and the release gate may delay a release; the hospital must accept this.
- **Assumed efficiencies:** 60%, 35% and 60% are estimates. If reviews are poor the saving is smaller; I would measure them in the repository and update the model.
- **Reviews as ritual:** reviews may become box-ticking; checklists and recorded defects keep them meaningful.
- **Tester–developer friction:** the tester must be part of the team and defects reported without blame.
- **Costs not counted:** patient harm and loss of trust are not in the figures, so the true benefit is greater.
- **Single release data:** the estimate rests on one release with 100 defects.

Is it workable for five developers?

Yes, if it stays lightweight. Extra developer effort is about 125 person-hours for the two reviews plus 80 hours for code review and unit tests, about 205 hours. Five developers over eight months provide 5 × 8 months × 20 days × 8 hours = 6,400 hours, so the plan uses about 3.2% of developer time, plus one tester. Reviews are short (90 minutes), checklist-driven and aimed at the highest-risk parts, and the repository tool can be free. The only large cost is the tester, who is also the biggest source of savings.

DAY-4 DESIGN CHALLENGE

Task 12 – Re-evaluate and Adapt the Quality Process 8 Marks

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wishes to release a new MediQueue version every two weeks rather than every six months.

In addition, MediQueue has to connect to the national digital health ID system, which an external agency maintains.

Patient health records will from now on be shared across hospitals.

Reassess your original solution in light of these changed conditions. You may: RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- What is different from the original situation
- Which assumptions in your quality improvement plan are impacted
- Why frequent releases alter the role of testing and of the tester
- Why sharing health records increases the severity of certain defect classes

Evidence Expected

A clear analysis of how the new conditions impact the original quality improvement plan.

What has changed

Table 24: Original situation compared with the changed situation

Aspect	Original situation	Changed situation
Release rhythm	One release roughly every six to eight months	A release every two weeks (26 a year, 13 times as frequent)
Interfaces	Own application plus an SMS provider	Also the national digital health ID system run by an external agency
Data	Appointment and prescription data of one hospital	Health records shared across hospitals

Aspect	Original situation	Changed situation
Control	The team controls all its code and releases	The agency decides when and how the ID system changes

Assumptions in my plan that are affected

- **One large review per release:** the plan assumes whole-document reviews (about 125 person-hours). This cannot be repeated every two weeks; reviews must be small and tied to each change.
- **Manual testing by one tester:** a full manual regression run (say 400 cases × 6 minutes = 40 person-hours) would use half of the 80 hours one tester has in a two-week cycle.
- **A test bed we control:** with an external ID system the test environment is the agency's sandbox, which may not behave like the live system.
- **A repository for one system:** it assumes every defect lies in our code, but some will now lie in the agency's system or in the contract between the two.
- **₹40,000 per production defect:** this rises when a defect can pass wrong data to other hospitals.

Why frequent releases change the role of testing and the tester

With a release every two weeks there is no "testing phase at the end". Testing must run continuously alongside development; a tester who joins only at the end becomes a bottleneck. The tester joins from story refinement onwards, spends less time on repetitive checks (which are automated and run on every change) and more on risk analysis, exploratory testing, test automation and the release decision. Every release must also be reversible and monitored, because a faulty release is live for 6,000 patients a day.

Why sharing health records raises the severity of some classes

Once records are shared, an I6-type defect (another patient's data in a document) is no longer a local error but a confidentiality breach across hospitals, with legal duties and possible harm to a patient treated on wrong information. Design and data defects (shared state, identity matching) and interface defects therefore become Critical, as would an access-control defect that was Medium before. Requirement defects about consent and identity also become Critical, while defects like I7 (manual) stay Low.

B. Evaluation of the Original Plan **2 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- Whether the planned reviews and test activities can fit within a two-week cycle
- Which new defect origins the external health ID system brings in
- Whether your defect repository is still adequate

Evidence Expected

A technically supported evaluation of how suitable the original plan is under the new conditions.

Does the original plan fit a two-week cycle?

Table 25: Fit of the planned activities to a two-week cycle

Planned activity	Fits in two weeks?	Reason / change required
Review of all rules (about 50 person-hours)	No, not as it stands	Review only the rules in that cycle's stories (30–45 minutes per story) and keep the rule table current
Design inspection (about 75 person-hours)	Partly	Do it only for stories touching shared data, identity or external interfaces; keep the checklist
Independent tester doing manual testing	No	Needs about 114 (original plan effort ÷ 13) + 40 (regression) + 30 (ID system and privacy checks) = 184 person-hours per cycle against about 140 available (tester 80 + 15% of the developers' 400 hours = 60): 31% over capacity
Release gate	Yes	Keep it; automate the checks that feed it
Code review and unit tests	Yes	Keep as part of the definition of done
Defect repository	Partly	Needs new fields (below)

The capacity figures are my estimates: the original plan's effort (reviews 125 + code review 80 + tester 8 months × 20 days × 8 hours = 1,280, total about 1,485 hours) divided by 13 for the smaller change in each release, plus one full regression run and the new checks.

New defect origins from the external health ID system

- **Contract and version defects:** the agency changes a field, version or rule without notice (the same pattern as I9).

- **Data-mapping and identity defects:** patients matched wrongly, inconsistent name or date formats, duplicate IDs.
- **Availability and speed:** the external service is slow or down and our application freezes or loses the booking.
- **Authentication and consent defects:** expired tokens, missing patient consent, wrong access rights.
- **Environment differences:** the agency's test system behaves differently from the live one.
- **Privacy and security defects:** records exposed in transit or in logs.

Is the defect repository still adequate?

No. It was designed for defects in the team's own system. It needs extra fields: whether the defect is internal, in the agency's system or in the interface; the interface version or contract reference; the agency's ticket number and response date; data sensitivity and the number of patients or hospitals affected; whether it must be reported to an authority; the release and pipeline stage where it was found; and the time taken to detect it and restore service. Without these the team cannot separate its own defects from the agency's or measure how fast problems are found and fixed.

C. Adaptation or Redesign **2 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to retain, modify or redesign the quality process.

If you modify or redesign it:

- Describe the revised process and the revised role of the tester
- Explain which testing principles gain greater importance now
- Explain how defects coming from the external system will be classified and tracked

If you keep the original plan, give adequate justification for why no change is needed.

Evidence Expected

A suitable response to the changed requirement with clear quality-engineering reasoning.

Decision: MODIFY the quality process

I **modify** the plan. I do not retain it unchanged, because Table 25 shows that its reviews and manual testing cannot fit a two-week cycle (about 31% over capacity) and because it has no

way to handle an external system. I do not redesign it from scratch, because the evidence it rests on is still sound: over half the defects begin in requirements and design, early detection is far cheaper (₹500 or ₹1,500 against ₹40,000), and an independent tester, a release gate and a repository are still needed. What changes is how often and how small each activity is, and what is automated.

The revised process

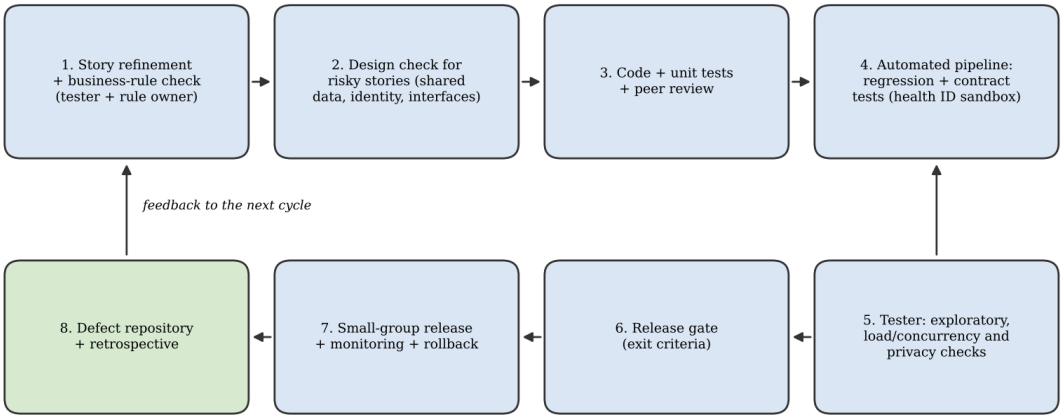


Figure 2: Revised quality cycle of two weeks

- **Story refinement:** every story gets acceptance criteria and a business-rule check by the tester and the hospital owner of the rule (30–45 minutes).
- **Design check:** only for high-risk stories (shared data, identity, external interface), using the checklist.
- **Development:** peer review and boundary-value unit tests are part of the definition of done.
- **Automated pipeline:** each change triggers automated regression tests on the critical paths (booking, token, fee, prescription, reminder) and contract tests against the health ID sandbox. The 40-hour manual regression run becomes an automated run of a few minutes.
- **Tester checks:** exploratory testing of changed stories, concurrency and load tests in the nightly run, and privacy checks for any change touching health records.
- **Release gate and safe rollout:** exit criteria are checked from pipeline results; the release goes first to a small group and can be rolled back; the SMS and health ID interfaces are monitored with alerts.

- **Retrospective:** a defect repository review each cycle to tune checklists and tests.

The tester's revised role

The tester becomes a quality coach and test engineer inside the team: analysing the risk of each story, building and maintaining the automated regression and contract tests, doing exploratory and privacy testing, owning the release-gate evidence and repository reports, and acting as contact point with the agency for interface problems. The tester is no longer only a final checker.

Principles that gain importance

- **Early testing and planning:** in two weeks there is no time for late correction.
- **Repeatable, reusable tests (regression):** every release must re-prove existing behaviour, so automation is essential.
- **Testing depends on context:** health data and money need stricter testing than the manual-based plan gave.
- **Defect clustering:** the interface and shared-data areas will produce most new defects.
- **Test invalid inputs and interfaces:** wrong, delayed or missing responses from the agency are normal cases that must be tested.

Classifying and tracking defects from the external system

Table 26: Additional classification for external-system defects

Item	How it is managed
Class	New "External interface" class with sub-classes: contract/version, data mapping, availability/performance, authentication/consent, environment difference
Owner	Internal (our adapter), agency (their service) or joint (contract unclear); decides who fixes it
Severity	Based on impact on patients and data; any wrong record or privacy exposure is Critical
Tracking	Our defect ID is linked to the agency's ticket; response times are logged; any workaround (queue, retry, manual process) is recorded
Measures	Time to detect, time to restore, number of such defects per release, share caused by agency changes
Prevention	Contract tests on each build, production monitoring, a named contact and a shared release calendar with the agency

D. Evidence-Based Defence **2 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Justify your final decision with evidence. Use:

- The defect data and the cost figures
- Defect origins and classes
- Release frequency
- Risks to patient safety and data privacy
- Team size and capacity

Your final decision must follow logically from the analysis you have carried out.

Evidence Expected

A well-supported final decision that brings together defect evidence, process reasoning and the changed constraints.

Defending the decision to modify

Table 27: Evidence supporting the decision to modify the plan

Evidence	What it shows	Effect on my decision
Defect data and cost	Patients found 60% of defects, which carry 91.9% of the cost (₹24,00,000 of ₹26,10,500); a production defect costs 80 times a requirement defect	Early detection stays, so the plan is not discarded
Origins and classes	50% of incidents and 52% of defects arise in requirements and design; coding is only 20% of incidents	Keep requirement and design checks, in small per-story form
Release frequency	26 releases a year; the original plan needs about 184 person-hours per cycle against about 140 available (31% over)	Unchanged plan would fail; automation and smaller reviews are needed
Patient safety and privacy	Shared records turn I6-type defects into breaches across hospitals (Critical)	Add design checks, privacy testing and a strict release gate
External agency	A new defect origin outside our control (contract, availability, mapping)	Add contract tests, monitoring and a repository class
Team size	Five developers (about 6,400 hours over eight months) plus one tester	Changes use about 3% of developer time plus the tester; a full redesign with a bigger test team is unaffordable

Reasoning. If the plan were kept unchanged, manual review and test effort would exceed capacity by 31%, and the team would either drop activities (returning to the past, when 60 defects escaped) or delay every release. If everything were redesigned, the parts that the case data strongly support would be thrown away at a cost a five-person team cannot bear. The modified plan keeps the proven elements, shrinks them to fit two weeks and automates the repeated checks.

Estimated benefit. With the effects assumed in Task 11 (about 20 defects reaching patients instead of 60 for the same amount of work) the saving was about ₹9 lakh per old-sized release. The same improvements are achievable at the new pace, and the release gate, monitoring and rollback limit the damage of any defect that still escapes to two weeks of exposure instead of six months.

Risks I accept and how I will watch them. The automated regression suite takes time to build, so early cycles will have lower coverage and the tester should start with the critical paths. The review and test efficiencies are estimates; the repository will show the real figures after about three cycles. The agency's behaviour is outside the team's control; contract tests and monitoring give early warning. If the repository shows that agency interface defects dominate and the team cannot control them, I would reconsider and redesign the process around a dedicated integration test group and a formal service agreement with the agency.